# Assignment 2A — Prompting as System Design

***Prompting Paradigms · Structured & Iterative Reasoning · Routing · 20 Marks***

| Component | Part | Marks |
|---|---|---|
| **Base Patterns** — Zero-shot, Few-shot, Instruction, Self-Critique, Function-Calling | Part A | 7 |
| **Reasoning & Custom Chains** — CoT, ToT, Self-Consistency, ReAct + 2 composed chains | Part B | 7 |
| **Prompt Router** — Classify Query → Route → Evaluate | Part C | 6 |
| **TOTAL** | | **20** |

Every prompting method is a sequence of productions — string manipulations applied to Q before the LLM samples A. This assignment implements them as composable Python functions (Part A), composes them into reasoning chains for domain tasks (Part B), and routes each query to the cheapest pattern that reaches acceptable accuracy (Part C).

*Q = query  A = answer  O = observation  C = critique  $\bar{A}$ = revised answer*

**Dependency ▸** Load your fine-tuned adapter (1A) or 4-bit quantized model (1B) as the LLM. Use instruction_dataset.jsonl for the 10 benchmark queries. No new data collection or training required.

## PART A — Base Patterns: Implement & Benchmark

*7 Marks · transformers · peft*

Implement five prompting patterns as Python functions sharing a uniform interface: each takes a query string and returns (answer, tokens_used, latency_sec). This makes patterns directly comparable and composable in Parts B and C.

**Recommended Libraries:** `transformers` `peft` `json` `time`

| Pattern | Production Sequence | LLM Calls | Key Idea |
|---|---|---|---|
| Zero-shot | `Q ──LLM──► A` | 1 | Direct — no augmentation |
| Few-shot (3-shot) | `Q → [examples + Q] ──LLM──► A` | 1 | Prepend k worked examples (in-context learning) |
| Instruction | `Q → [role + steps + Q] ──LLM──► A` | 1 | Explicit role + step-by-step format |
| Self-Critique | `Q →A →C →Ā (3 calls)` | 3 | Generate → critique → revise |
| Function-Calling | `Q ──LLM──► JSON(schema)` | 1 | Structured output to a fixed JSON schema |

*Shared setup — run once before implementing patterns:*
*Load your Assignment-1 adapter → wrap as generate(prompt) → str.*
*Load 10 queries + known answers from instruction_dataset.jsonl as the benchmark set.*
*Fix decoding (greedy if from 1A; your best config from 1B) and max_new_tokens=200.*

### Step A1 — Zero-shot, Few-shot & Instruction [2 Marks]

Three single-call patterns. Zero-shot sends the query directly; Few-shot prepends 3 worked examples from instruction_dataset.jsonl; Instruction wraps the query in an explicit role + numbered-step template. Same call, richer input.

### Step A2 — Self-Critique (3-Production Chain) [2 Marks]

Three sequential LLM calls: generate an answer, critique it, then revise using the critique. This is the foundational prompt chain — the output of call N becomes the input of call N+1. Print the intermediate answer and the critique for 2 queries.

### Step A3 — Function-Calling / Structured Output [1 Mark]

Define a JSON schema for your domain output (e.g., {entity, value, unit, source_clause}). Use JSON mode or an output parser to force valid JSON. Show one parse-failure case and how you handle it (constrained decoding or a retry prompt).

### Step A4 — Benchmark: 10 Queries × 5 Patterns [2 Marks]

Run all five patterns on the same 10 queries. Score accuracy 0–2 (0 = wrong, 1 = partial, 2 = correct). Record tokens and latency.

**Write 75 words:** which pattern gives the best accuracy-to-token-cost ratio? Does Self-Critique justify its 3× token cost over Zero-shot?

## PART B — Reasoning & Custom Chains

*7 Marks · transformers · peft*

Add reasoning-augmented patterns, then compose Part A + reasoning patterns into 2 domain chains. Use the 10 benchmark queries plus 2 additional multi-step domain problems that require reasoning.

| Technique | When to Use in Your Domain |
|---|---|
| Chain-of-Thought (CoT) | Any multi-step domain query |
| Tree-of-Thought (ToT) | Complex problem with alternatives |
| Self-Consistency | Reduce reasoning variance (test-time compute) |
| ReAct | Query needing a tool call or a calculation |

### Step B1 — Structured Reasoning: CoT, ToT & Self-Consistency [3 Marks]

Implement the three structured-reasoning techniques on 2 multi-step domain problems. For ToT, show the 3 candidate paths and the scoring; for Self-Consistency, show the 5 samples and the majority vote. Compare against a plain single-call answer in a table.

### Step B2 — Iterative Reasoning: ReAct [2 Marks]

**ReAct —** implement a Thought → Action → Observation loop (≥ 3 cycles) on a domain question that needs an external step. Each Action calls a local tool/function (e.g., a calculator, or your A3 function-calling output) and the Observation feeds the next Thought. Print the full trace and the final answer.

### Step B3 — Two Custom Chains [2 Marks]

Compose Part A + Part B patterns into 2 domain chains. Each targets a distinct query type and must justify why the composition beats any single pattern alone.

### Chain 1 — Plan → Solve → Verify

```
Q ──LLM──► [plan / decompose] → solve ──LLM──► A ──LLM──► Ā (Self-Critique)
```

*Best for: high-stakes queries where a wrong answer is costly.*

### Chain 2 — Decompose → Solve-each → Synthesise (Socratic)

```
Q ──LLM──► [sub-Q1, sub-Q2] → solve each → [A1, A2] ──LLM──► Ā
```

*Best for: multi-part queries a single pass answers incompletely.*

Apply each chain to 2 queries and compare against a single-pattern baseline in a table.

## PART C — Prompt Router: Classify → Route → Evaluate

***6 Marks · transformers***

Build an intelligent router that classifies each query and dispatches it to the cheapest pattern that achieves acceptable accuracy, then evaluate the router against an always-zero-shot baseline.

### Step C1 — Query Classifier & Routing Table [2 Marks]

One zero-shot LLM call classifies the query into a type and returns exactly one label — no explanation. Map each type to a pattern from Parts A/B:

| Query Type | Route To |
|---|---|
| Simple factual query | Zero-shot |
| Query needing worked examples | Few-shot |
| Structured extraction task | Function-Calling |
| Multi-step reasoning problem | Chain-of-Thought |
| Complex problem with alternatives | Tree-of-Thought |
| High-variance / needs consensus | Self-Consistency |
| Task needing a tool call or calculation | ReAct |
| High-stakes / verification needed | Plan → Solve → Verify chain |

Test the classifier on 5 queries; report accuracy; refine the classifier prompt if accuracy < 3/5.

### Step C2 — Router Dispatch & Evaluation [2 Marks]

Implement the router function: classify → dispatch to the selected pattern → return (answer, route, tokens, latency). Run it on the 10 benchmark queries and write your observations.

### Step C3 — Cost vs Quality Analysis [2 Marks]

Compare the router against always-zero-shot (reuse the Part A numbers — already benchmarked).

**Write 100 words:** Which pattern did the router select most often? Where did routing beat the zero-shot baseline? Did chaining justify its extra token cost? Which single pattern would you deploy in production, and why?

## Submission Deliverables

Please submit all the relevant .ipynb and corresponding html file with outputs.